

COSMO® Cube — Firmware Architecture v1.0

Document: COSMO-FIRMWARE-v1.md

Platform: ESP32-S3 (ESP-IDF 5.2)

Language: C / C++17

Author: Vadim Marmeladov, Inventor

Date: 2026-06-12

1. Development Environment Setup

1.1 Prerequisites

```
# Install ESP-IDF 5.2 (macOS / Linux)
mkdir -p ~/esp && cd ~/esp
git clone -b v5.2 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf && ./install.sh esp32s3
source export.sh
```

```
# Verify
idf.py --version
# Expected: ESP-IDF v5.2.x
```

1.2 Repository Structure

```
cosmo-firmware/
% % % main/
%   % % % main.c           # Entry point, task init
%   % % % app_config.h     # Hardware pin definitions
%   % % % wifi/
%   %   % % % wifi_manager.c # WiFi init, reconnect logic
%   %   % % % wifi_manager.h
%   % % % % api/
%   %   % % % lot_api.c      # LOT API HTTP client
%   %   % % % lot_api.h
%   %   % % % lot_api_cert.c # TLS certificate bundle
%   % % % % display/
%   %   % % % ssd1327_driver.c # SSD1327 SPI driver
%   %   % % % ssd1327_driver.h
%   %   % % % ui_screens.c    # Screen state machine
%   %   % % % ui_screens.h
%   % % % % sensors/
%   %   % % % bme280_driver.c  # BME280 I2C driver
%   %   % % % icm42688_driver.c # ICM-42688 SPI driver
%   %   % % % apds9960_driver.c # APDS-9960 I2C driver
%   %   % % % sensors.h
%   % % % % camera/
%   %   % % % camera_hal.c    # ESP32-Camera HAL wrapper
%   %   % % % camera_hal.h
%   % % % % button/
%   %   % % % button_handler.c # Copy button ISR + debounce
%   %   % % % button_handler.h
%   % % % % power/
%   %   % % % power_manager.c  # BQ25892 I2C control, sleep
%   %   % % % power_manager.h
%   % % % % ota/
%   %   % % % ota_manager.c    # Secure OTA update
%   %   % % % ota_manager.h
%   % % % % storage/
%   %   % % % nvs_config.c     # NVS for API key, WiFi creds
%   %   % % % nvs_config.c
% % % components/
%   % % % % esp32-camera/     # Espressif camera component
%   % % % % partition_table.csv # Custom partition layout
%   % % % % sdkconfig.defaults # Build configuration
%   % % % % CMakeLists.txt
% % % % README.md
```

1.3 Build & Flash

```
cd cosmo-firmware
idf.py set-target esp32s3
idf.py menuconfig # Configure WiFi credentials (for initial dev)
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

2. Hardware Pin Assignments

2.1 Pin Map (ESP32-S3-MINI-1U)

```
// app_config.h

// --- Display (SSD1327 SPI) ---
#define DISPLAY_MOSI    GPIO_NUM_11
#define DISPLAY_CLK    GPIO_NUM_12
#define DISPLAY_CS      GPIO_NUM_10
#define DISPLAY_DC      GPIO_NUM_9
#define DISPLAY_RST     GPIO_NUM_8

// --- Camera (DVP parallel) ---
#define CAM_D0          GPIO_NUM_14
#define CAM_D1          GPIO_NUM_13
#define CAM_D2          GPIO_NUM_21
#define CAM_D3          GPIO_NUM_47
#define CAM_D4          GPIO_NUM_48
#define CAM_D5          GPIO_NUM_45
#define CAM_D6          GPIO_NUM_38
#define CAM_D7          GPIO_NUM_39
#define CAM_VSYNC       GPIO_NUM_6
#define CAM_HREF        GPIO_NUM_7
#define CAM_PCLK        GPIO_NUM_15
#define CAM_XCLK        GPIO_NUM_16

// --- I2C Bus (BME280, APDS-9960, BQ25892) ---
#define I2C_SDA         GPIO_NUM_1
#define I2C_SCL         GPIO_NUM_2
#define I2C_FREQ_HZ     400000

// --- SPI Bus (ICM-42688) ---
#define IMU_MOSI        GPIO_NUM_35
#define IMU_MISO        GPIO_NUM_36
#define IMU_CLK         GPIO_NUM_37
#define IMU_CS          GPIO_NUM_34

// --- Button (Copy) ---
#define BTN_COPY        GPIO_NUM_0 // GPIO0, internal pull-up
#define BTN_DEBOUNCE_MS 50

// --- LED (RGB) ---
#define LED_R           GPIO_NUM_3
#define LED_G           GPIO_NUM_4
#define LED_B           GPIO_NUM_5

// --- Power ---
#define CHG_INT         GPIO_NUM_17 // BQ25892 interrupt pin
#define VSYS_EN         GPIO_NUM_18 // System power enable
```

3. Firmware Architecture

3.1 Task Architecture (FreeRTOS)

```

% % % % % % % % % % % % % % % %
%   app_main()   %
%   (init + tasks) %
% % % % % % % % % % % % % % % %
%
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
%
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
%   wifi_task   %   %   display_task   %   %   sensor_task   %
%   (Core 0)   %   %   (Core 1)   %   %   (Core 0)   %
%   Priority 5 %   %   Priority 4 %   %   Priority 3 %
%   Stack 8KB %   %   Stack 4KB   %   %   Stack 4KB   %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
%   api_task   %   %   %   button_task   %
%   (Core 0)   %   %   %   (Core 1, ISR) %
%   Priority 4 %   %   %   Priority 6   %
%   Stack 12KB %   %   %   Stack 2KB   %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
%   Event Queue %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
%   ui_event_queue %
%   (xQueueCreate) %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %

```

3.2 Main Event Loop Flow

```
// main.c – simplified
void app_main(void) {
    // 1. Hardware init
    nvs_flash_init();
    power_manager_init();
    display_init();
    sensors_init();
    button_init();
    camera_init();

    // 2. Show boot screen
    ui_show_boot_screen();

    // 3. Start WiFi (non-blocking, shows status on display)
    wifi_manager_start();

    // 4. Create event queue
    ui_event_queue = xQueueCreate(10, sizeof(ui_event_t));

    // 5. Spawn FreeRTOS tasks
    xTaskCreatePinnedToCore(wifi_task,      "wifi",      8192,  NULL, 5, NULL, 0);
    xTaskCreatePinnedToCore(api_task,       "api",       12288, NULL, 4, NULL, 0);
    xTaskCreatePinnedToCore(display_task,   "display",   4096,  NULL, 4, NULL, 1);
    xTaskCreatePinnedToCore(sensor_task,    "sensors",   4096,  NULL, 3, NULL, 0);
    xTaskCreatePinnedToCore(ota_task,       "ota",       8192,  NULL, 2, NULL, 0);
}
```

4. LOT API Client

4.1 Notification Polling (api_task)

```

// lot_api.c

#define LOT_API_BASE      "https://lot-systems.com"
#define LOT_NOTIFICATIONS "/api/hardware/notifications"
#define LOT_LOG_ENDPOINT  "/api/hardware/log"
#define POLL_INTERVAL_SEC 60

typedef struct {
    char message[128];
    char source[32];
    char timestamp[32];
    int type;           // 0=info, 1=alert, 2=reminder
} lot_notification_t;

static void api_task(void *pvParams) {
    while (1) {
        if (wifi_is_connected()) {
            lot_notification_t notif = {0};
            esp_err_t err = lot_api_get_notification(&notif);

            if (err == ESP_OK && strlen(notif.message) > 0) {
                ui_event_t event = {
                    .type = UI_EVENT_NOTIFICATION,
                    .data = notif
                };
                xQueueSend(ui_event_queue, &event, 0);
            }
        }
        vTaskDelay(pdMS_TO_TICKS(POLL_INTERVAL_SEC * 1000));
    }
}

```

4.2 Copy Button !' LOT Log (lot_api_post_log)

```

// lot_api.c

esp_err_t lot_api_post_log(const sensor_data_t *sensors, const char *image_b64) {
    char body[1024];
    snprintf(body, sizeof(body),
        "{\n"
        "\"device\": \"%s\", \"\n"
        "\"serial\": \"%s\", \"\n"
        "\"timestamp\": \"%s\", \"\n"
        "\"temperature\": %.2f, \"\n"
        "\"humidity\": %.2f, \"\n"
        "\"pressure\": %.2f, \"\n"
        "\"light\": %d, \"\n"
        "\"event\": \"copy_button\"\n"
        "}",
        DEVICE_NAME, DEVICE_SERIAL,
        get_iso8601_time(),
        sensors->temperature,
        sensors->humidity,
        sensors->pressure,
        sensors->light_lux
    );

    esp_http_client_config_t config = {
        .url = LOT_API_BASE LOT_LOG_ENDPOINT,
        .method = HTTP_METHOD_POST,
        .cert_pem = lot_ca_cert_pem,    // Pinned TLS cert
        .timeout_ms = 5000,
    };

    esp_http_client_handle_t client = esp_http_client_init(&config);
    esp_http_client_set_header(client, "Content-Type", "application/json");
    esp_http_client_set_header(client, "Authorization", nvs_get_api_key());
    esp_http_client_set_post_field(client, body, strlen(body));

    esp_err_t err = esp_http_client_perform(client);
    int status = esp_http_client_get_status_code(client);
    esp_http_client_cleanup(client);

    if (err == ESP_OK && status == 200) {
        led_pulse(LED_G, 2);    // Green: success
        return ESP_OK;
    } else {
        led_pulse(LED_R, 3);    // Red: failed
        return ESP_FAIL;
    }
}

```

5. Display Driver

5.1 SSD1327 SPI Driver (ssd1327_driver.c)

```

// SSD1327 initialization sequence
static const uint8_t ssd1327_init_cmds[] = {
    0xAE,          // Display OFF
    0xA0, 0x53,    // Remap (col addr 0!'seg 127, vertical addr increment)
    0xA1, 0x00,    // Display start line = 0
    0xA2, 0x00,    // Display offset = 0
    0xA4,          // Normal display (not all on/off)
    0xA8, 0x7F,    // Multiplex ratio = 128 (1/128 duty)
    0xB1, 0x51,    // Phase length (phase1=1, phase2=5)
    0xB3, 0x01,    // Display clock div = 1, osc freq = 0
    0xAB, 0x01,    // Enable internal VDD regulator
    0xB6, 0x01,    // Second precharge period = 1 DCLK
    0xBE, 0x07,    // VCOMH deselect level
    0xBC, 0x08,    // Precharge voltage = Vcc × 0.5
    0xD5, 0x62,    // Function selection B
    0xAF,          // Display ON
};

void ssd1327_write_text(const char *text, uint8_t x, uint8_t y, uint8_t scale) {
    // Render text at pixel position using embedded 5×7 bitmap font
    // scale: 1 = 5×7px, 2 = 10×14px, 3 = 15×21px
}

void ssd1327_show_notification(const lot_notification_t *n) {
    ssd1327_clear();
    ssd1327_draw_logo(2, 2, 20, 20);          // LOT® logo top-left
    ssd1327_write_text(n->message, 4, 28, 2); // Message, scaled 2×
    ssd1327_write_text(n->source, 4, 110, 1);  // Source label, small
    ssd1327_refresh();
}

```

6. Sensor Integration

6.1 BME280 Weather Sensor

```

// sensors.c
typedef struct {
    float temperature; // Celsius
    float humidity;    // %RH
    float pressure;     // hPa
    int   light_lux;    // from APDS-9960
    int   proximity;    // from APDS-9960
    float accel_x;      // g
    float accel_y;
    float accel_z;
} sensor_data_t;

static void sensor_task(void *pvParams) {
    bme280_init();
    icm42688_init();
    apds9960_init();

    sensor_data_t data = {0};
    TickType_t last_wake = xTaskGetTickCount();

    while (1) {
        bme280_read_all(&data.temperature, &data.humidity, &data.pressure);
        apds9960_read_light(&data.light_lux, &data.proximity);
        icm42688_read_accel(&data.accel_x, &data.accel_y, &data.accel_z);

        // Update shared sensor state
        xSemaphoreTake(sensor_mutex, portMAX_DELAY);
        memcpy(&g_sensor_data, &data, sizeof(sensor_data_t));
        xSemaphoreGive(sensor_mutex);

        // Auto-sleep display when proximity < 5 and light < 10 lux
        if (data.proximity < 5 && data.light_lux < 10) {
            ui_event_send(UI_EVENT_SLEEP);
        }

        vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(5000)); // 5s sample rate
    }
}

```

7. Button Handler

7.1 Copy Button ISR + Action

```
// button_handler.c

static void IRAM_ATTR btn_isr_handler(void *arg) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xSemaphoreGiveFromISR(btn_semaphore, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

static void button_task(void *pvParams) {
    while (1) {
        if (xSemaphoreTake(btn_semaphore, portMAX_DELAY)) {
            vTaskDelay(pdMS_TO_TICKS(BTN_DEBOUNCE_MS));

            // Confirm button still pressed (debounce confirm)
            if (gpio_get_level(BTN_COPY) == 0) {
                // Trigger: capture sensor snapshot + camera frame
                sensor_data_t snap;
                sensors_get_snapshot(&snap);

                // Optional camera capture (user-configured)
                char *img_b64 = NULL;
                if (nvs_get_bool("camera_on_copy")) {
                    img_b64 = camera_capture_b64();
                }

                // Show "Logging..." on display
                ui_event_send(UI_EVENT_LOG_PENDING);

                // POST to LOT API
                esp_err_t result = lot_api_post_log(&snap, img_b64);

                if (result == ESP_OK) {
                    ui_event_send(UI_EVENT_LOG_SUCCESS);
                } else {
                    ui_event_send(UI_EVENT_LOG_FAIL);
                }

                free(img_b64);
            }
        }
    }
}
```

8. Power Management

8.1 Deep Sleep / Light Sleep Policy

Condition	Action	Wakeup Trigger
Battery > 20%, display off > 30s	Light sleep	Button press, timer (60s)
Battery < 10%	Deep sleep	Qi charging detected
Charging complete	Wake, show 100%	N/A
WiFi failed 5×	Airplane mode, deep sleep	Button press

```
// power_manager.c

void power_enter_light_sleep(uint32_t duration_sec) {
    // Configure wakeup sources
    esp_sleep_enable_timer_wakeup((uint64_t)duration_sec * 1000000);
    esp_sleep_enable_gpio_wakeup(); // Button
    gpio_wakeup_enable(BTN_COPY, GPIO_INTR_LOW_LEVEL);
    gpio_wakeup_enable(CHG_INT, GPIO_INTR_LOW_LEVEL); // Charger detect

    // Light sleep – WiFi suspended, CPU halted, RAM retained
    esp_light_sleep_start();

    // On wakeup: resume WiFi if needed
    esp_sleep_wakeup_cause_t cause = esp_sleep_get_wakeup_cause();
    if (cause == ESP_SLEEP_WAKEUP_TIMER) {
        // Timer wakeup: poll API, then return to sleep
        lot_api_check_notifications();
        power_enter_light_sleep(60);
    }
}
```

9. OTA Firmware Update

9.1 Partition Layout (partition_table.csv)

#	Name,	Type,	SubType,	Offset,	Size,	Flags
nvs,	data,	nvs,		0x9000,	0x6000,	
otadata,	data,	ota,		0xf000,	0x2000,	
ota_0,	app,	ota_0,		0x20000,	0x1C0000,	
ota_1,	app,	ota_1,		0x1E0000,	0x1C0000,	
storage,	data,	spiffs,		0x3A0000,	0x60000,	

9.2 OTA Update Flow

```
// ota_manager.c

#define OTA_URL "https://lot-systems.com/api/hardware/firmware/latest"

void ota_check_and_update(void) {
    esp_http_client_config_t config = {
        .url = OTA_URL,
        .cert_pem = lot_ca_cert_pem,
        .timeout_ms = 30000,
        .buffer_size = 4096,
    };

    esp_https_ota_config_t ota_config = {
        .http_config = &config,
    };

    // Check version header before downloading
    // X-Firmware-Version response header != compare with FIRMWARE_VERSION
    // Only update if remote version > local

    esp_err_t ret = esp_https_ota(&ota_config);
    if (ret == ESP_OK) {
        ui_show_message("Updated. Restarting.");
        vTaskDelay(pdMS_TO_TICKS(2000));
        esp_restart();
    }
}
```

10. Security

10.1 Secure Boot + Flash Encryption

```
# Enable in sdkconfig.defaults
CONFIG_SECURE_BOOT=y
CONFIG_SECURE_BOOT_V2_ENABLED=y
CONFIG_SECURE_BOOT_SIGNING_KEY="secure_boot_key.pem"
CONFIG_FLASH_ENCRYPTION_ENABLED=y
CONFIG_FLASH_ENCRYPTION_MODE_RELEASE=y
```

10.2 API Key Storage (NVS Encrypted)

```
// nvs_config.c

// API key stored in encrypted NVS partition at factory
// Key: "lot_api_key", Namespace: "cosmo_config"

esp_err_t nvs_get_api_key(char *out_key, size_t len) {
    nvs_handle_t h;
    nvs_open("cosmo_config", NVS_READONLY, &h);
    esp_err_t err = nvs_get_str(h, "lot_api_key", out_key, &len);
    nvs_close(h);
    return err;
}
```

10.3 TLS Certificate Pinning

```
// lot_api_cert.c
// Embed lot-systems.com CA certificate at compile time
const char lot_ca_cert_pem[] = \
"-----BEGIN CERTIFICATE-----\n"
// (certificate contents embedded at factory build time)
"-----END CERTIFICATE-----\n";
```

11. Factory Provisioning Flow

1. Flash firmware (idf.py flash)
2. Run provisioning tool:
python3 tools/provision.py \
--port /dev/ttyUSB0 \
--serial CQ-001-26 \
--api-key <LOT-generated-device-key> \
--wifi-ssid <factory-test-ssid> \
--wifi-pass <factory-test-pass>
3. Device boots, connects to LOT API, registers serial
4. LOT site confirms device activated
5. QA: press Copy button !' verify Log tab entry appears on lot-systems.com
6. QA: trigger test notification from LOT admin !' verify display shows message
7. Wipe WiFi credentials (user provisions via BLE app on first use)
8. Seal enclosure

12. Build Versioning

Field	Value
Firmware version format	MAJOR.MINOR.PATCH-BUILD
Example	1.0.0-001
Version stored in	NVS + displayed in BLE advertisement
OTA check on boot	Yes (if WiFi connected within 30s of boot)
Rollback	Automatic (ESP-IDF OTA rollback on crash within 60s)

Document v1.0 — COSMO® CIA — LOT Systems, Inc.

Inventor: Vadim Marmeladov — 2026-06-12

